



TicketZone

Sistema de Vendas de Ingressos

Documentação Técnica & Tutorial Completo

Versão 1.0 · Android Studio 2023.1.1 · Java · SQLite

2025

■ SUMÁRIO

- 1. Visão Geral do Sistema**
 - 1.1 Objetivo
 - 1.2 Tecnologias utilizadas
 - 1.3 Funcionalidades principais
- 2. Arquitetura do Projeto**
 - 2.1 Estrutura de pastas
 - 2.2 Camadas da aplicação
 - 2.3 Fluxo de dados
- 3. Banco de Dados**
 - 3.1 Tabelas e colunas
 - 3.2 Relacionamentos
 - 3.3 Como visualizar o banco
- 4. Tutorial de Instalação e Configuração**
 - 4.1 Pré-requisitos
 - 4.2 Importando no Android Studio
 - 4.3 Sincronizando o Gradle
 - 4.4 Rodando no emulador
- 5. Tutorial das Telas**
 - 5.1 Tela de Login
 - 5.2 Tela de Cadastro
 - 5.3 Tela Principal — Lista de Ingressos
 - 5.4 Tela de Formulário — Criar/Editar
 - 5.5 Busca de ingressos
- 6. Referência das Classes Java**
 - 6.1 DatabaseHelper
 - 6.2 Models: Ticket e User
 - 6.3 Activities
 - 6.4 TicketAdapter
- 7. Guia de Extensão**
 - 7.1 Adicionar novo campo
 - 7.2 Adicionar nova tela
 - 7.3 Boas práticas de segurança

1. VISÃO GERAL DO SISTEMA

1.1 Objetivo

O **TicketZone** é um aplicativo Android para gerenciamento e venda de ingressos de eventos. Ele permite que usuários cadastrados façam login, cadastrem eventos com seus respectivos ingressos, controlem o estoque de ingressos disponíveis, realizem buscas e editem ou removam registros, tudo integrado a um banco de dados SQLite local no dispositivo.

1.2 Tecnologias utilizadas

Plataforma

■ **Android (API 24+)**

Linguagem

■ **Java**

Banco de dados

■■ **SQLite (nativo Android)**

UI

■ **Material Components 1.10**

IDE

■■ **Android Studio 2023.1.1**

Build

■ **Gradle 8.1 / AGP 8.1.1**

1.3 Funcionalidades principais

- Autenticação de usuários (cadastro e login com validação)
- Sessão persistida via SharedPreferences — mantém login entre sessões
- CRUD completo de ingressos: **C**riar, **L**er, **A**tualizar, **D**eletar
- Busca em tempo real por nome do evento ou local
- Controle de estoque com badge de status (Disponível / Poucos / Esgotado)
- Confirmação de exclusão para evitar deleções acidentais
- Interface com tema roxo/dourado inspirada em ticketeiras profissionais
- Layout responsivo compatível com diferentes tamanhos de tela

2. ARQUITETURA DO PROJETO

2.1 Estrutura de pastas

```

app/src/main/ ■■■ java/com/example/ticketapp/ ■ ■■■ activities/ ■ ■ ■■■
LoginActivity.java ← Tela de login ■ ■ ■■■ RegisterActivity.java ← Cadastro de
usuário ■ ■ ■■■ MainActivity.java ← Lista de ingressos ■ ■ ■■■
TicketFormActivity.java ← Criar / editar ingresso ■ ■■■ adapters/ ■ ■ ■■■
TicketAdapter.java ← RecyclerView adapter ■ ■■■ database/ ■ ■ ■■■
DatabaseHelper.java ← SQLiteOpenHelper ■ ■■■ models/ ■ ■■■ User.java ← Modelo de
usuário ■ ■■■ Ticket.java ← Modelo de ingresso ■■■ res/ ■■■ layout/ ← Arquivos XML
de interface ■■■ drawable/ ← Ícones vetoriais e shapes ■■■ menu/ ← Menu da toolbar
■■■ mipmap-*/ ← Ícones do launcher ■■■ values/ ← Cores, strings, temas
  
```

2.2 Camadas da aplicação

O projeto segue uma arquitetura em três camadas simples, adequada para aplicações Android didáticas:

Camada	Responsabilidade	Classes
Apresentação (UI)	Exibir dados e capturar interações	Activities + Layouts XML
Lógica de negócio	Validações e regras de uso	Activities + Adapter
Dados	Persistência e consultas	DatabaseHelper + Models

2.3 Fluxo de dados

1

Usuário interage com a tela (Activity)

Ex.: toca no botão 'Salvar'

2

Activity valida os campos do formulário

Verifica campos obrigatórios e formatos

3

Activity chama DatabaseHelper

Passa um objeto Model como parâmetro

4

DatabaseHelper executa SQL no SQLite

INSERT, UPDATE, DELETE ou SELECT

5

Resultado retorna para a Activity

true/false ou List<Ticket>

6

Activity atualiza a interface

RecyclerView, Toast ou navegação

3. BANCO DE DADOS

3.1 Tabelas e colunas

Tabela: users

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK AUTOINCREMENT	Identificador único
name	TEXT	NOT NULL	Nome completo
email	TEXT	UNIQUE NOT NULL	E-mail de login
password	TEXT	NOT NULL	Senha do usuário

Tabela: tickets

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK AUTOINCREMENT	Identificador único
event_name	TEXT	NOT NULL	Nome do evento
location	TEXT		Local do evento
event_date	TEXT		Data do evento (texto)
category	TEXT		Categoria (Show/Esporte/etc)
price	REAL	NOT NULL	Valor do ingresso (R\$)
quantity	INTEGER	NOT NULL	Qtd de ingressos disponíveis

3.2 Relacionamentos

Nesta versão, as tabelas **users** e **tickets** são independentes. O usuário logado é identificado apenas pela sessão (SharedPreferences). Para uma versão futura com multiusuário, recomenda-se adicionar a coluna **user_id** **INTEGER REFERENCES users(id)** na tabela tickets.

■ **Nota:** O banco de dados fica salvo em `/data/data/com.example.ticketapp/databases/TicketApp.db` no dispositivo.

3.3 Como visualizar o banco de dados

Método 1 — App Inspection (recomendado)

1

Rode o app no emulador

Clique em Run  no Android Studio

2

Abra o App Inspection

View → Tool Windows → App Inspection

3

Clique em Database Inspector

Selecione com.example.ticketapp

4

Explore as tabelas

Clique em users ou tickets para ver os dados

Método 2 — ADB + DB Browser for SQLite

```
# No terminal do Android Studio: adb shell run-as com.example.ticketapp \ cp
/data/data/com.example.ticketapp/databases/TicketApp.db /sdcard/ adb pull
/sdcard/TicketApp.db . # Abra TicketApp.db no DB Browser for SQLite (sqlitebrowser.org)
```

Método 3 — Logcat (debug rápido)

```
List tickets = dbHelper.getAllTickets(); for (Ticket t : tickets) { Log.d("DB",
t.getId() + " | " + t.getEventName() + " | R$" + t.getPrice()); } // Filtre o Logcat
pela tag: DB
```

4. TUTORIAL DE INSTALAÇÃO E CONFIGURAÇÃO

4.1 Pré-requisitos

Requisito	Versão mínima	Como obter
Android Studio	2023.1.1 (Hedgehog)	developer.android.com/studio
JDK	11 ou superior	Já incluso no Android Studio
Android SDK	API 24 (Android 7.0)	SDK Manager no Android Studio
Emulador	API 24+ recomendado	AVD Manager no Android Studio
RAM	8 GB recomendado	Para rodar emulador com fluidez

4.2 Importando o projeto no Android Studio

- Extraia o ZIP baixado**
Escolha uma pasta de fácil acesso, ex: C:\Projetos\TicketApp

- Abra o Android Studio**
Na tela inicial, clique em 'Open'

- Selecione a pasta**
Navegue até a pasta extraída e clique em OK

- Aguarde a indexação**
O Android Studio vai indexar os arquivos (1-2 min)

4.3 Sincronizando o Gradle

Após abrir o projeto, o Android Studio exibirá uma barra amarela no topo com a mensagem *"Gradle files have changed since last project sync"*. Clique em **Sync Now**. Caso não apareça, vá em **File** → **Sync Project with Gradle Files**.

■ **Nota:** Se aparecer o erro 'android.useAndroidX not enabled', verifique se o arquivo `gradle.properties` na raiz do projeto contém as linhas: `android.useAndroidX=true` e `android.enableJetifier=true`

4.4 Rodando no emulador

- Abra o AVD Manager**
Tools → Device Manager → Create Device

2

Escolha um dispositivo

Ex: Pixel 6 | API 34 | x86_64

3

Inicie o emuladorClique no botão  verde ao lado do dispositivo

4

Execute o appClique em Run  na barra superior do Android Studio

5

Primeiro uso

Cadastre um usuário e faça login para explorar o app

5. TUTORIAL DAS TELAS

5.1 Tela de Login

- Ao abrir o app, o LoginActivity é iniciado.
- Se já houver uma sessão salva, o app redireciona direto para a lista de ingressos.
- Preencha **E-mail** e **Senha** e toque em **ENTRAR**.
- Caso as credenciais estejam incorretas, um Toast exibe 'E-mail ou senha inválidos'.
- Para criar conta, toque em **Cadastre-se** no rodapé da tela.

5.2 Tela de Cadastro

- Preencha **Nome**, **E-mail**, **Senha** e **Confirmação de senha**.
- Validações automáticas: e-mail no formato correto, senha com mínimo de 6 caracteres.
- E-mails duplicados são detectados antes de salvar no banco.
- Após o cadastro, o app retorna automaticamente para a tela de login.

5.3 Tela Principal — Lista de Ingressos

- Exibe todos os ingressos cadastrados em cards com nome do evento, local, data e preço.
- Badge colorido indica o status: **verde** = disponível, **laranja** = poucos, **vermelho** = esgotado.
- Toque no ícone  para editar um ingresso.
- Toque no ícone  para excluir — aparece um diálogo de confirmação.
- Toque no **botão + (FAB)** no canto inferior direito para adicionar novo ingresso.
- Toque no ícone  na toolbar para abrir a busca.
- Menu superior () contém a opção de **Sair** (logout).

5.4 Tela de Formulário — Criar/Editar

- Ao criar, todos os campos começam vazios. Ao editar, os dados do ingresso são carregados.
- Campos: **Nome do evento** (obrigatório), **Local**, **Data**, **Categoria**, **Preço** (obrigatório), **Quantidade** (obrigatório).
- Toque em **SALVAR** para confirmar ou **CANCELAR** para voltar sem salvar.
- Após salvar, o app retorna para a lista e exibe um Toast de confirmação.

5.5 Busca de ingressos

- Toque no ícone  na toolbar da tela principal.
- Digite qualquer parte do nome do evento ou do local.
- A lista é filtrada em **tempo real** conforme você digita.
- Para voltar a ver todos os ingressos, apague o texto ou feche a busca com **X**.

6. REFERÊNCIA DAS CLASSES JAVA

6.1 DatabaseHelper

Localização: **database/DatabaseHelper.java** | Herda de: SQLiteOpenHelper

Método	Parâmetros	Retorno	Descrição
registerUser()	User user	boolean	Insere novo usuário no banco
loginUser()	String email, String password	User ou null	Autentica usuário
emailExists()	String email	boolean	Verifica e-mail duplicado
addTicket()	Ticket ticket	boolean	Insere novo ingresso
getAllTickets()	—	List<Ticket>	Retorna todos os ingressos
getTicketById()	int id	Ticket ou null	Busca ingresso por ID
updateTicket()	Ticket ticket	boolean	Atualiza dados do ingresso
deleteTicket()	int id	boolean	Remove ingresso pelo ID
searchTickets()	String query	List<Ticket>	Busca por nome ou local

6.2 Models: Ticket e User

User.java

```
// Campos
private int id;
private String name; // Nome completo
private String email; // E-mail (único)
private String password; // Senha
// Construtores disponíveis
User() // Vazio
User(String name, String email, String password) // Novo usuário
User(int id, String name, String email, String password) // Com ID (do banco)
```

Ticket.java

```
// Campos
private int id;
private String eventName; // Nome do evento
private String location; // Local do evento
private String eventDate; // Data (String: dd/MM/yyyy)
private String category; // Categoria
private double price; // Preço unitário
private int quantity; // Qtd em estoque
```

6.3 Activities

Classe	Descrição	Inicia quem?
LoginActivity	Tela inicial. Verifica sessão ao abrir.	App (LAUNCHER)
RegisterActivity	Formulário de cadastro de novo usuário.	LoginActivity
MainActivity	Lista de ingressos com busca e CRUD.	LoginActivity (após login)
TicketFormActivity	Formulário criar/editar ingresso.	MainActivity

6.4 TicketAdapter

O **TicketAdapter** é um `RecyclerView.Adapter` responsável por inflar o layout *item_ticket.xml* para cada ingresso na lista. Implementa a interface **OnTicketClickListener** com dois callbacks:

```
public interface OnTicketClickListener { void onEditClick(Ticket ticket); // Botão
editar void onDeleteClick(Ticket ticket); // Botão excluir } // A MainActivity
implementa essa interface e recebe os eventos: @Override public void onEditClick(Ticket
ticket) { Intent intent = new Intent(this, TicketFormActivity.class);
intent.putExtra("ticketId", ticket.getId()); startActivity(intent); }
```

7. GUIA DE EXTENSÃO

7.1 Como adicionar um novo campo ao ingresso

1

Adicione o campo no modelo

Ticket.java: novo atributo + getter/setter

2

Atualize o banco de dados

DatabaseHelper: adicione a coluna no CREATE TABLE

3

Atualize os métodos SQL

addTicket(), updateTicket() e cursor de leitura

4

Adicione o campo no layout

activity_ticket_form.xml: novo TextInputLayout

5

Leia e salve na Activity

TicketFormActivity: leia o campo e passe ao model

6

Exiba no card (opcional)

item_ticket.xml + TicketAdapter.onBindViewHolder

7.2 Como adicionar uma nova tela

Siga sempre este padrão para manter a consistência do projeto:

```
// 1. Crie o arquivo Java em activities/ public class MinhaNovaActivity extends
AppCompatActivity { @Override protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState); setContentView(R.layout.activity_minha_nova); } }
// 2. Crie o layout XML em res/layout/activity_minha_nova.xml // 3. Registre no
AndroidManifest.xml: // 4. Navegue até ela de outra Activity: startActivity(new
Intent(this, MinhaNovaActivity.class));
```

7.3 Boas práticas de segurança

- **Hash de senha:** O app atual armazena senhas em texto puro. Em produção, use SHA-256 ou BCrypt antes de salvar no banco.
- **Validação de entrada:** Sempre valide e sanitize os campos antes de inserir no banco para evitar SQL Injection.
- **SharedPreferences:** Não armazene dados sensíveis (tokens, senhas) em SharedPreferences. Prefira o EncryptedSharedPreferences.
- **Backup:** Configure android:allowBackup="false" no Manifest se o banco contiver dados sensíveis.
- **ProGuard:** Habilite a ofuscação de código (minifyEnabled true) para builds de produção.

TicketZone — Documentação gerada automaticamente · Versão 1.0 · 2025 · Desenvolvido com Android Studio, Java e SQLite.